



CI1056 – ALGORITMOS E ESTRUTURA DE DADOS II

Profa. Rachel Reis
rachel@inf.ufpr.br

AULA DE HOJE

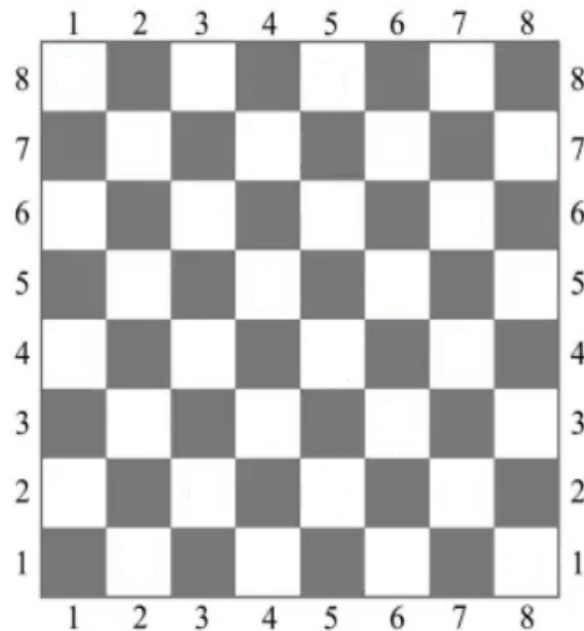
Backtracking e o Problema das 8 Rainhas

Adaptação do material gentilmente
disponibilizado pelo Prof. Daniel Oliveira

MOTIVAÇÃO

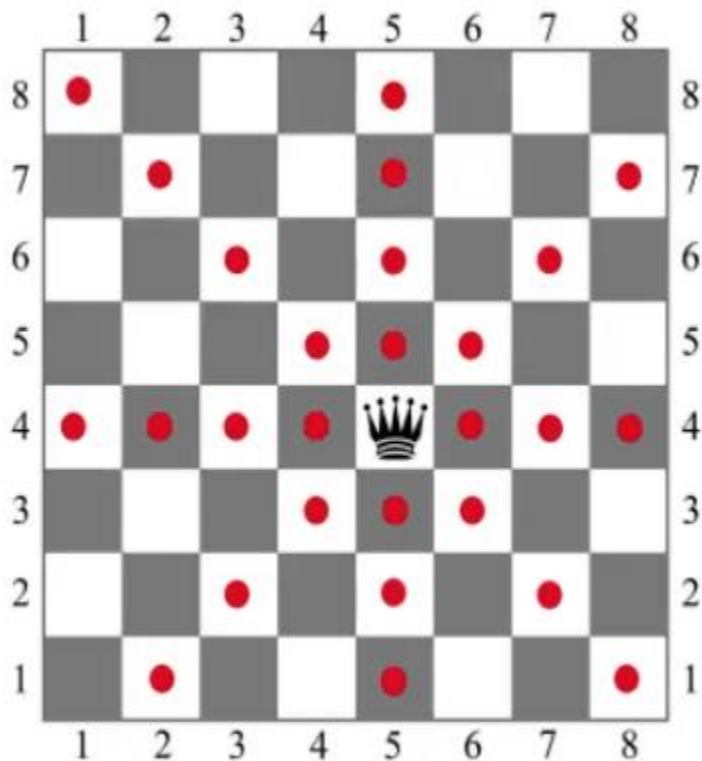
PROBLEMA DAS 8 RAINHAS

- Como colocar 8 rainhas em um tabuleiro de xadrez de forma que nenhuma rainha **ataque** outra rainha?



MOTIVAÇÃO

PROBLEMA DAS 8 RAINHAS

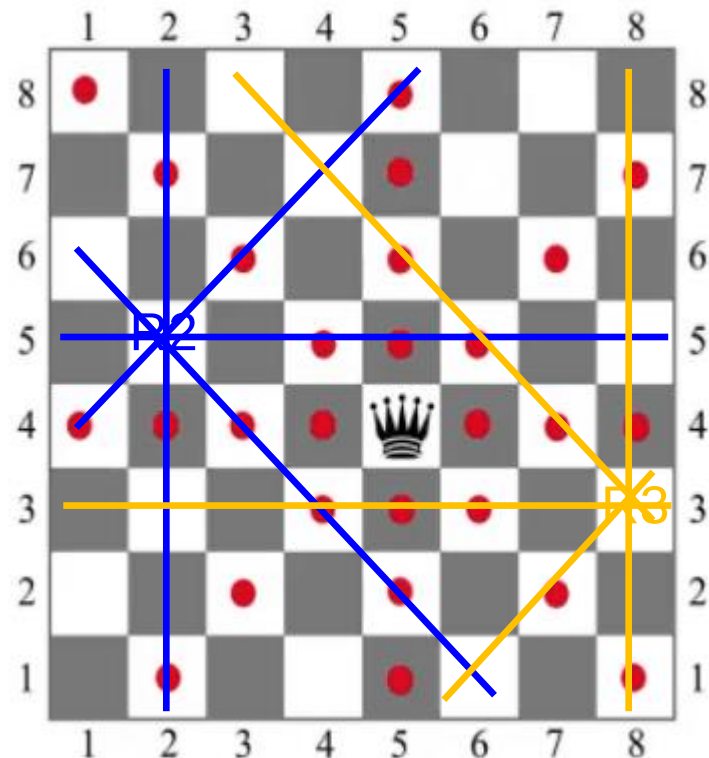
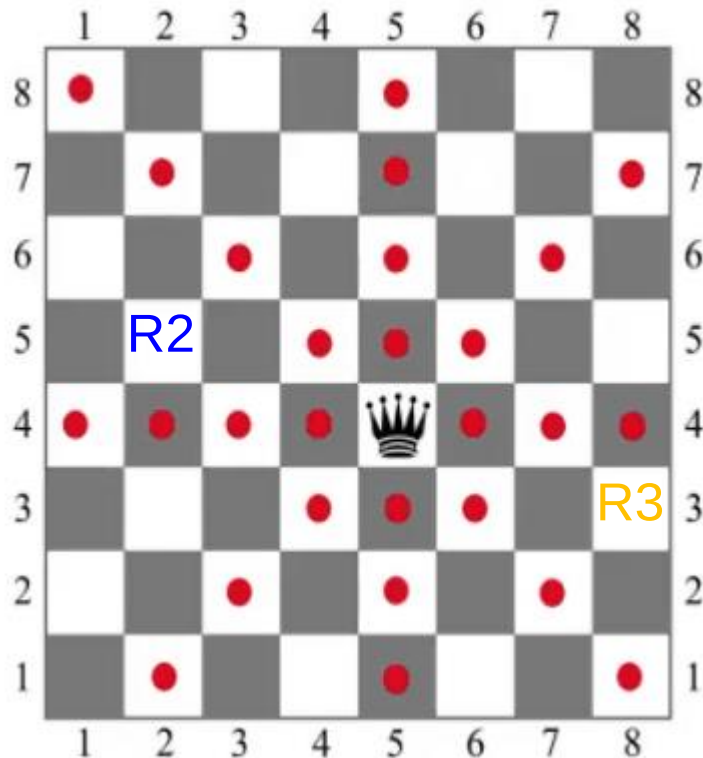


- Rainhas se movimentam, e capturam peças, em toda a linha, coluna, diagonal e anti-diagonal.
- Os pontos em vermelho representam as casas em que a rainha pode se movimentar/atacar.

MOTIVAÇÃO

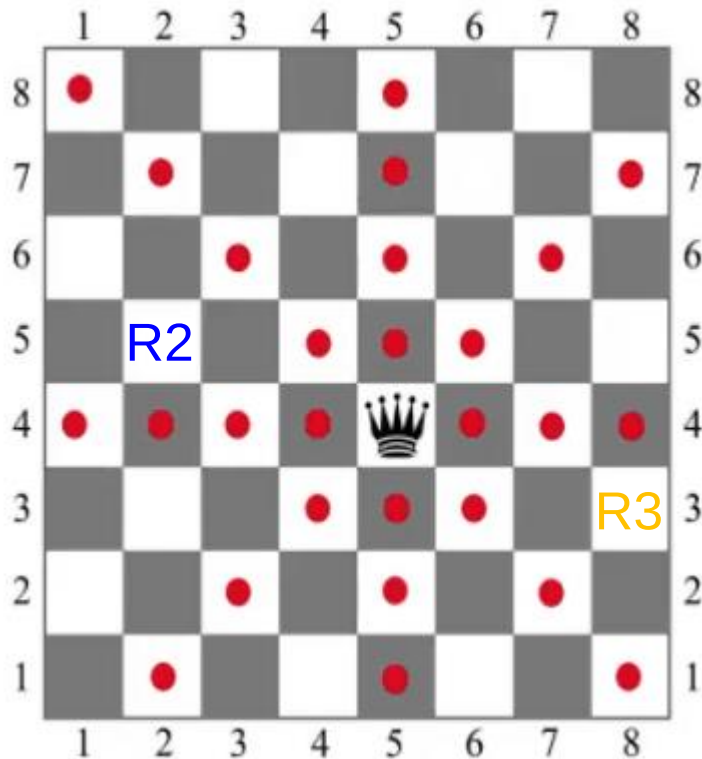
PROBLEMA DAS 8 RAINHAS

- Posição e movimento de novas rainhas no tabuleiro.



MOTIVAÇÃO

PROBLEMA DAS 8 RAINHAS

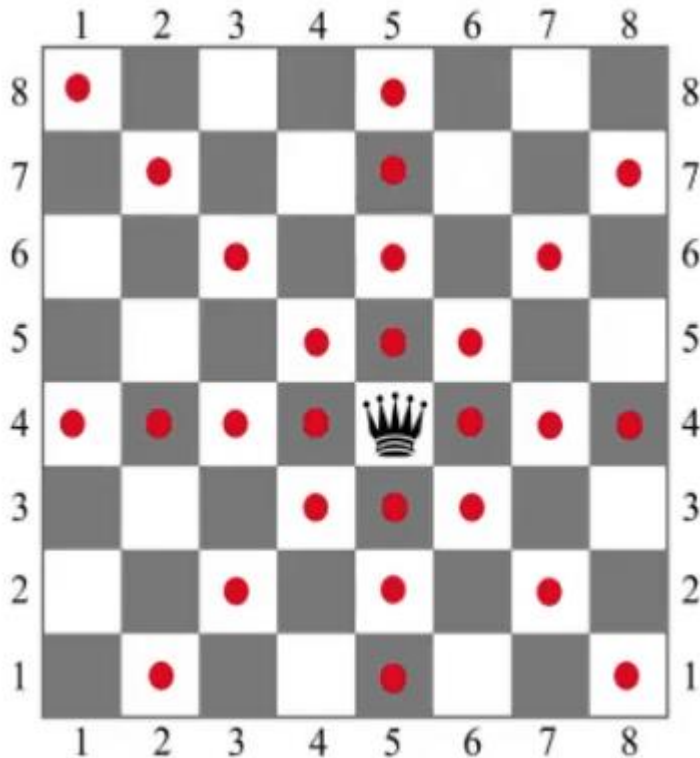


Objetivo:

Descobrir **todas** as maneiras de colocar 8 rainhas no tabuleiro de forma que elas **não** se ataquem.

MOTIVAÇÃO

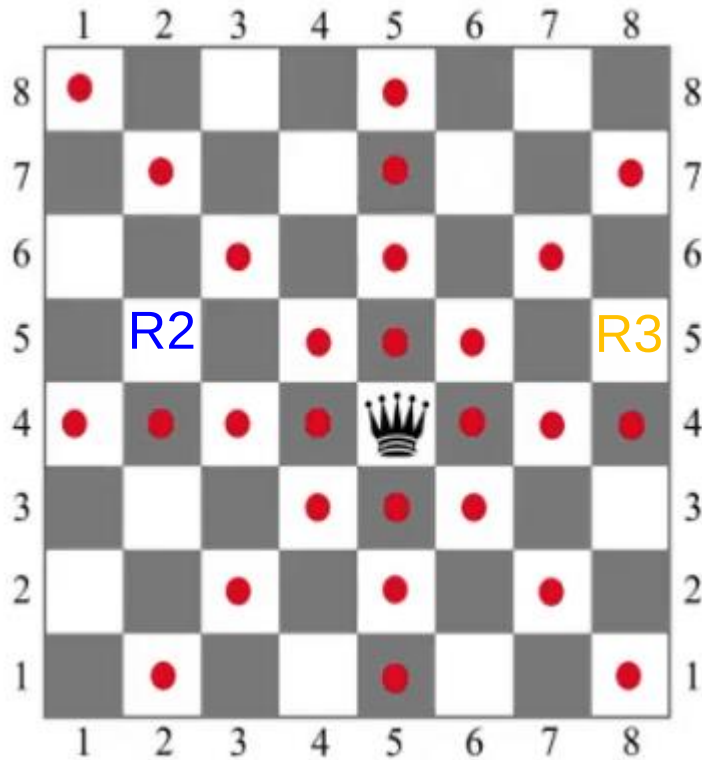
PROBLEMA DAS 8 RAINHAS



- Verificar se uma rainha na posição (i, j) ataca outra é trivial:
 - linha: i
 - coluna: j
 - diagonal: $i + j$
 - anti-diagonal: $i - j$

MOTIVAÇÃO

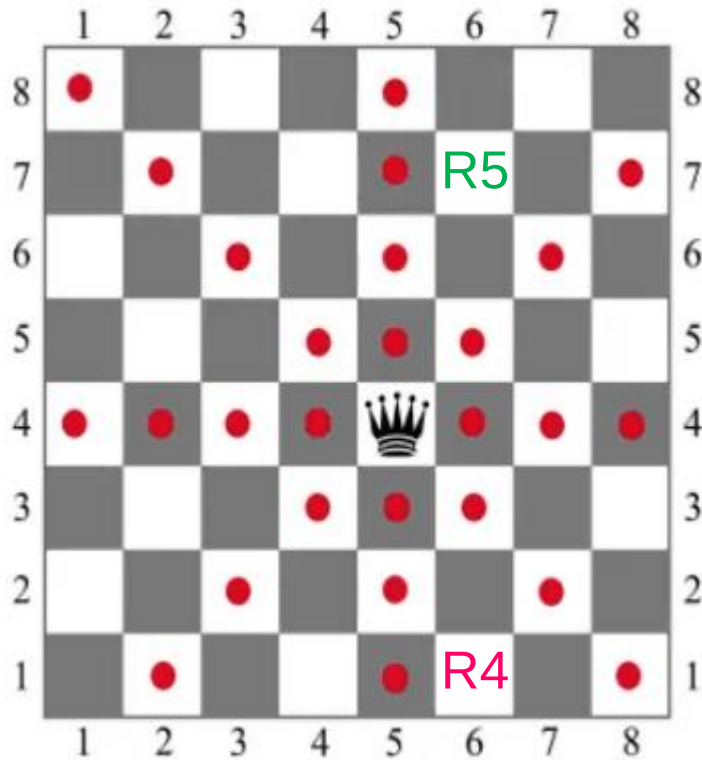
PROBLEMA DAS 8 RAINHAS



- Exemplo 1: as rainhas **R2** e **R3** compartilham a mesma linha.

MOTIVAÇÃO

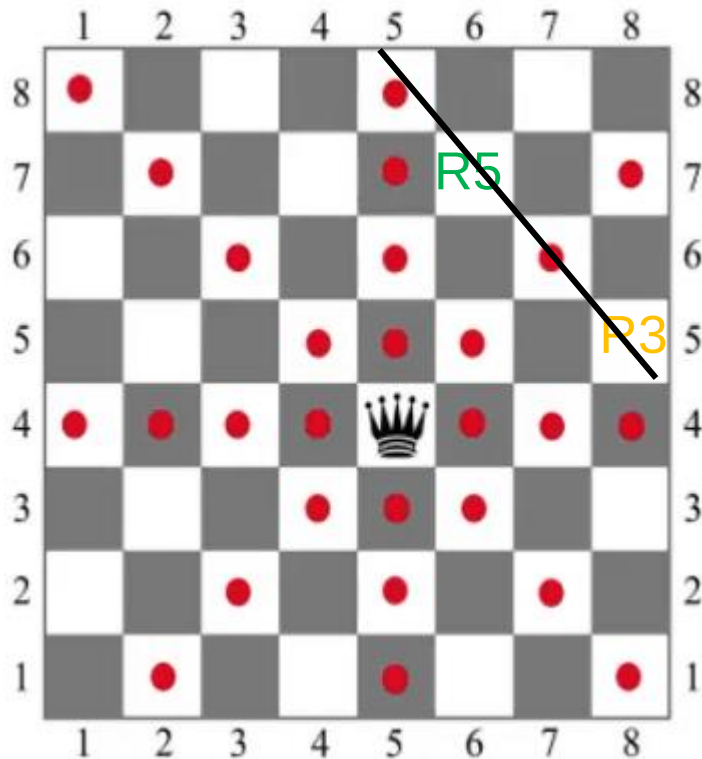
PROBLEMA DAS 8 RAINHAS



- Exemplo 2: as rainhas **R4** e **R5** compartilham a mesma coluna.

MOTIVAÇÃO

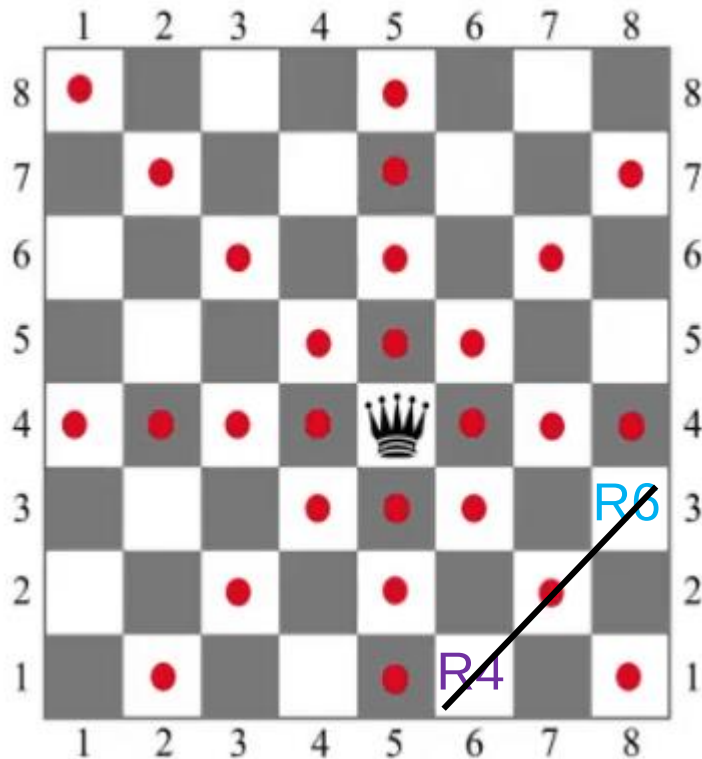
PROBLEMA DAS 8 RAINHAS



- Exemplo 3: as rainhas **R3** e **R5** estão na mesma diagonal.
- diagonal: $i + j$
 - R3** $5 + 8 = 13$
 $6 + 7 = 13$
 - R5** $7 + 6 = 13$
 $8 + 5 = 13$

MOTIVAÇÃO

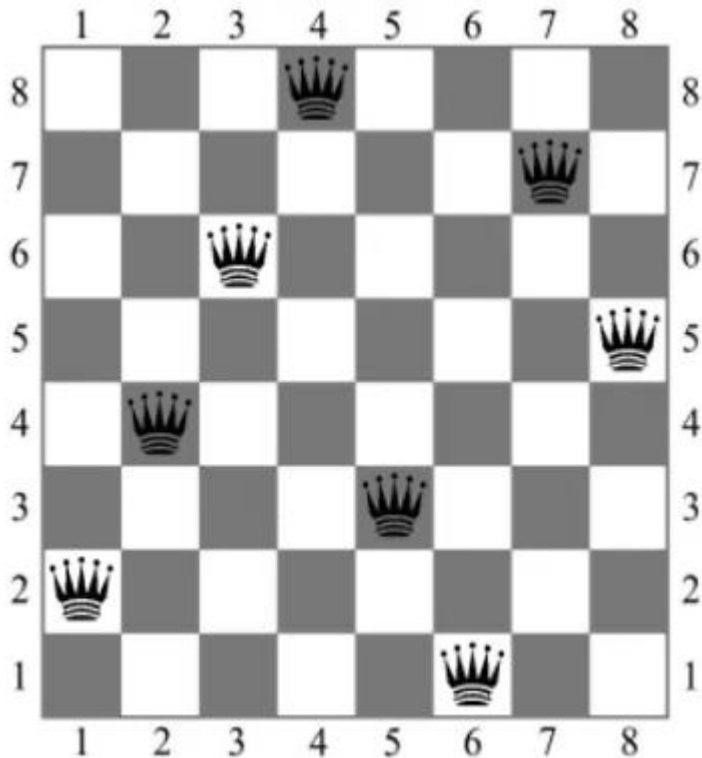
PROBLEMA DAS 8 RAINHAS



- Exemplo 4: as rainhas **R4** e **R6** estão na mesma anti-diagonal.
- anti-diagonal: $i - j$
 - R4** $1 - 6 = -5$
 - $2 - 7 = -5$
 - R6** $3 - 8 = -5$

MOTIVAÇÃO

PROBLEMA DAS 8 RAINHAS



- O problema das 8 rainhas possui solução, conforme exemplo ao lado.
- Queremos todas as soluções e não apenas uma.

PROBLEMA DAS N-RAINHAS

- Problema computacional

n-rainhas

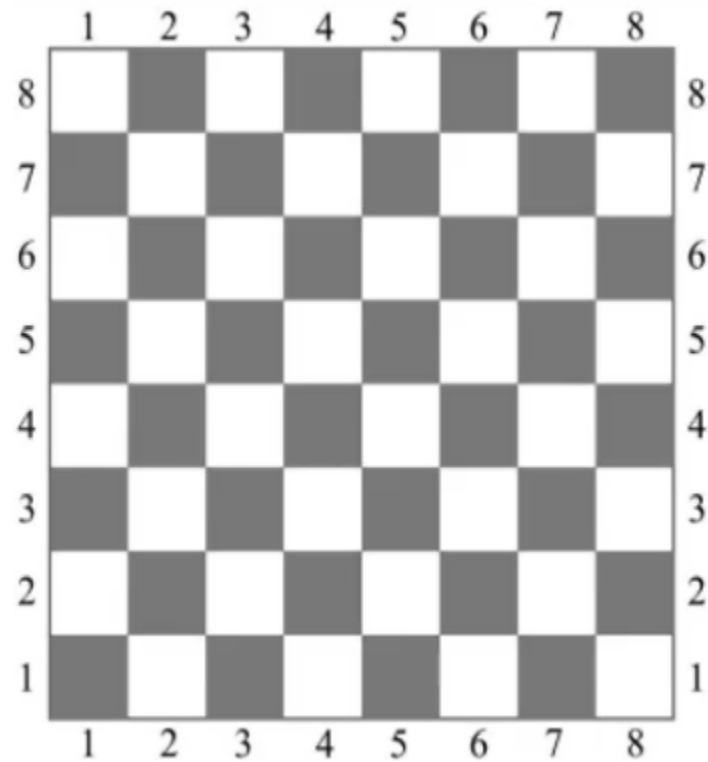
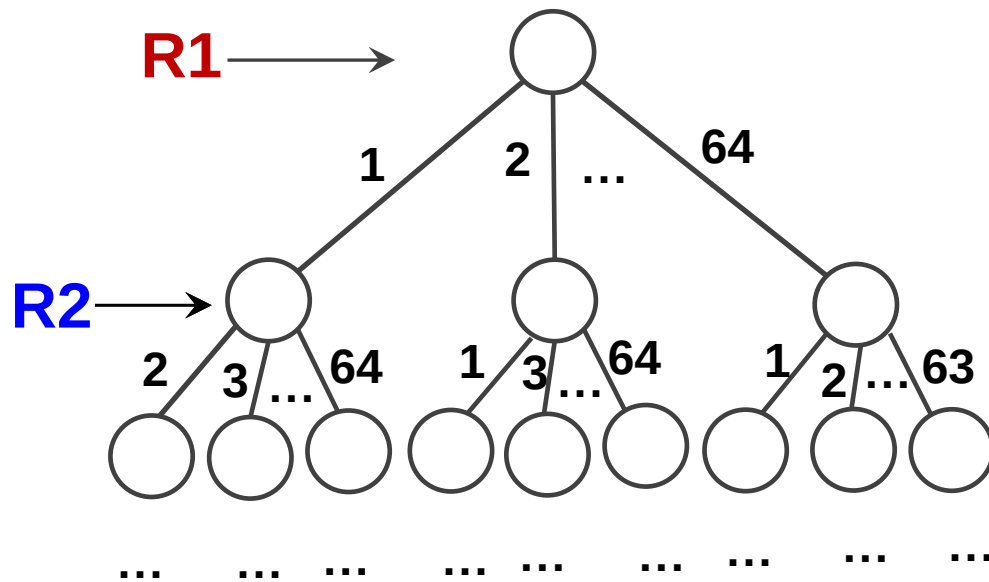
Instância: n , onde n é a quantidade de rainhas colocadas em um tabuleiro $n \times n$.

Resposta: todas as soluções onde as n rainhas não se atacam.

8-RAINHAS – PROPOSTA 1

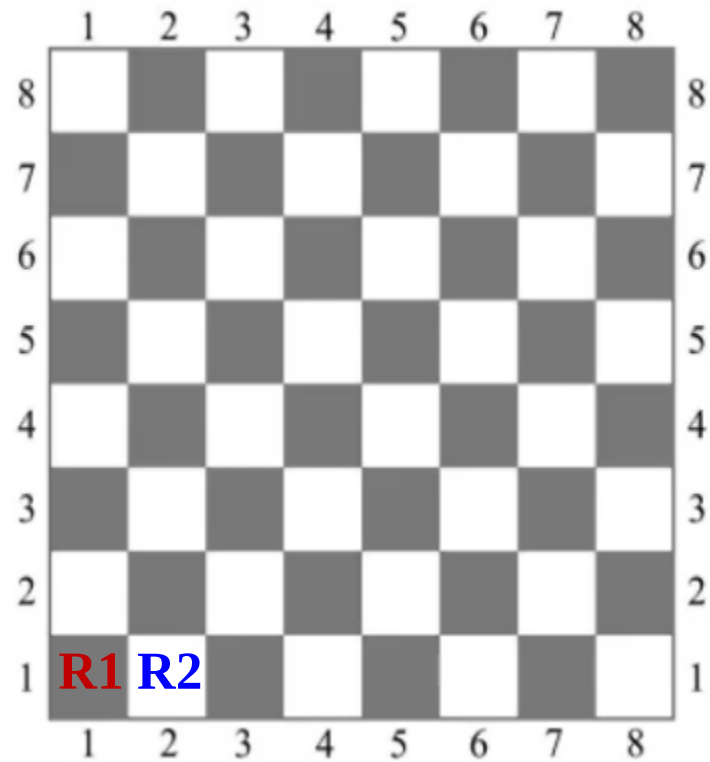
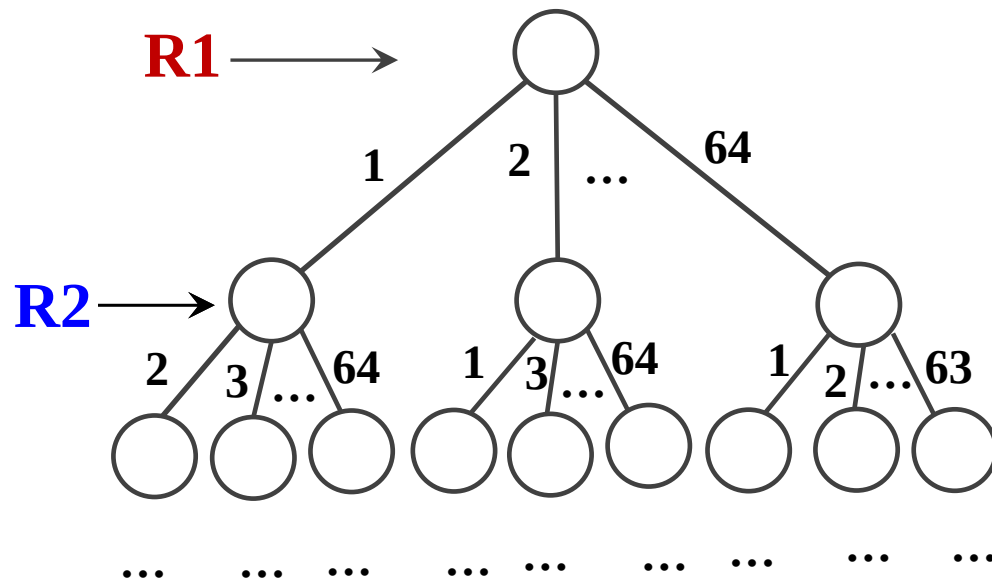
- Testar **todas as combinações** possíveis e verificar quais respeitam as condições.

Árvore de decisão



- A rainha **R1** pode ser colocada em qualquer uma das posições do tabuleiro. Ex. 1 – (1,1), 2 – (1,2) e assim por diante.
- A rainha **R2** tem 63 opções de posição em cada subárvore.

Árvore de decisão



- Alguns ramos da árvores são equivalentes.
- Ex.: **R1**(1,1) e **R2**(1,2) é equivalente a **R2**(1,1) e **R1**(1,2).

8-RAINHAS – PROPOSTA 1

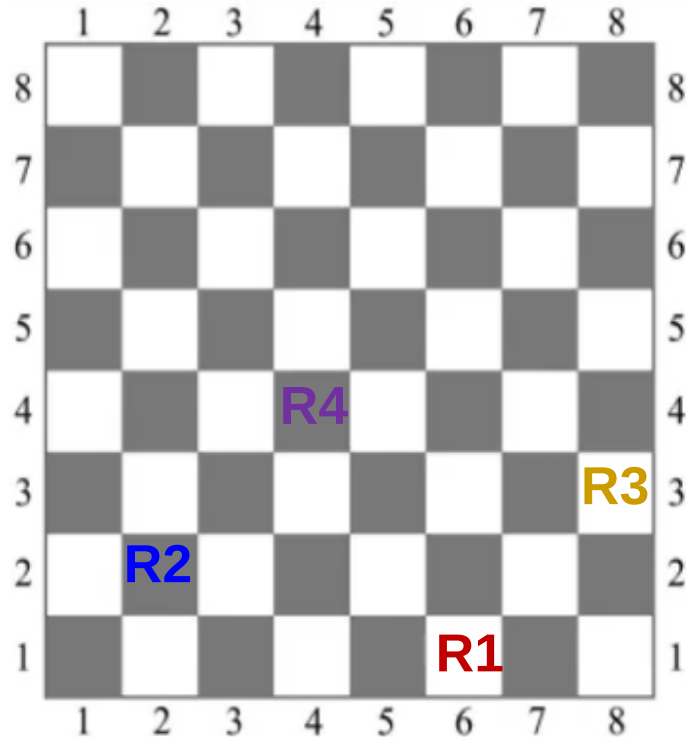
- Testar todas as **combinações** possíveis e verificar quais respeitam as condições.
- Quantas combinações diferentes existem?

$$\binom{64}{8} = 4.426.165.368$$

Solução não viável!

8-RAINHAS – PROPOSTA 2

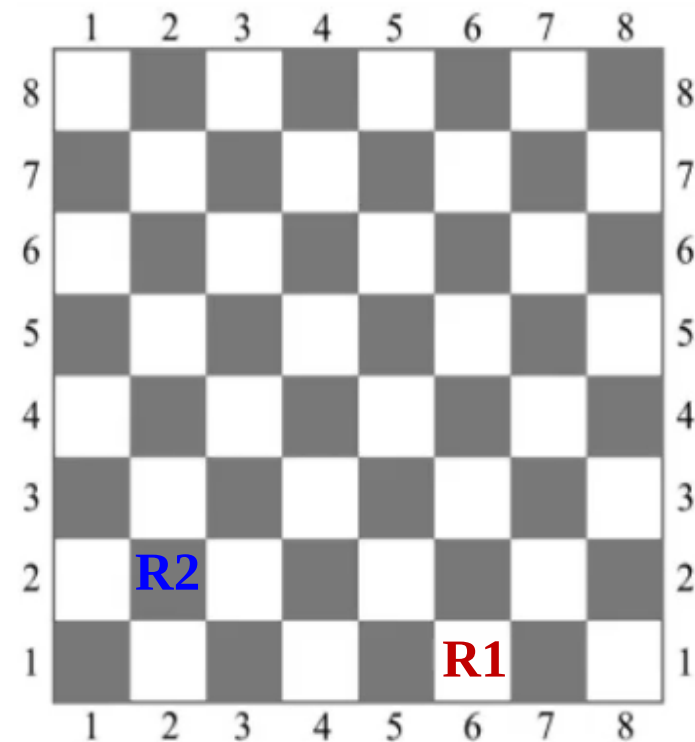
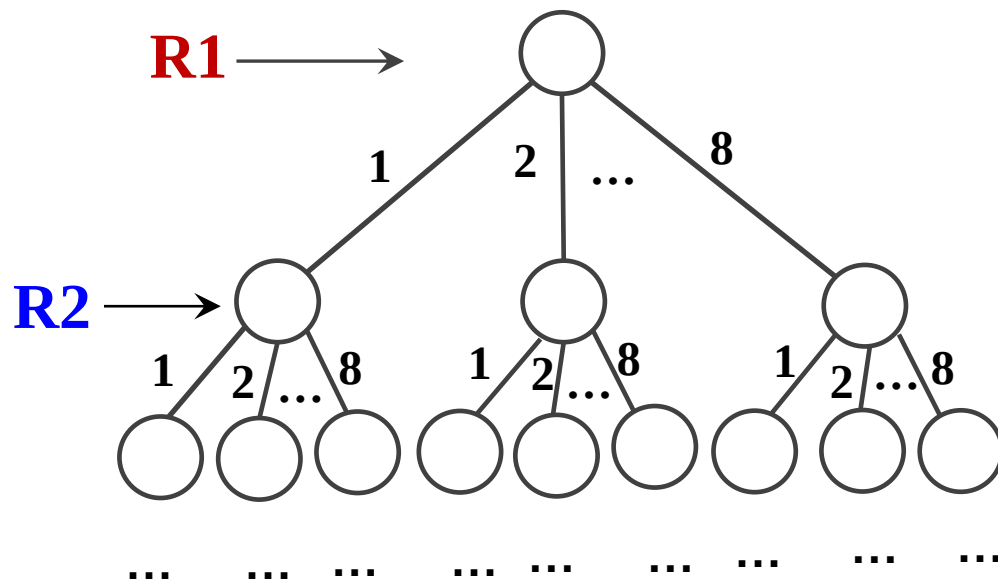
- Vimos que colocar mais de uma rainha na mesma linha **não** é viável.



O que fazer?

- Nenhuma solução pode ter mais de uma rainha na mesma linha.
- Testar todas as combinações com uma rainha por linha.

Árvore de decisão



- A rainha **R1** deve ser colocada na primeira linha e em qualquer coluna. Ex. 1 – (1,1), 2 – (1,2), ... 8 – (1,8).
- A rainha **R2** deve ser colocada na segunda linha e em qualquer coluna. Ex. 1 – (2,1), 2 – (2,2), ... 8 – (2,8).

8-RAINHAS – PROPOSTA 2

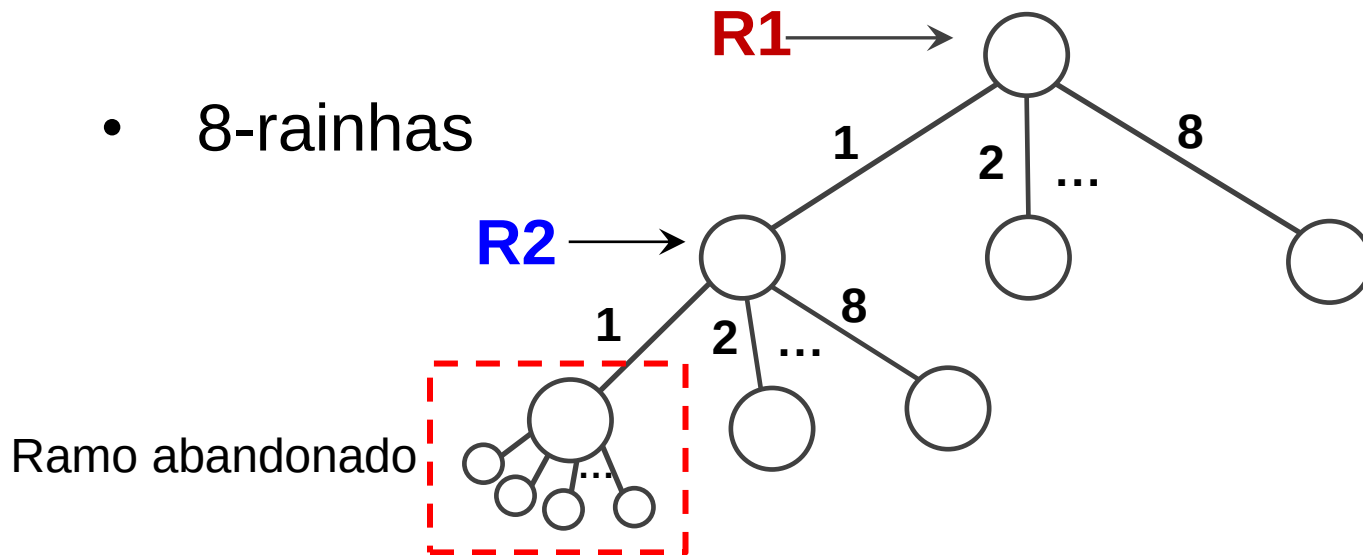
- Nenhuma solução pode ter mais de uma rainha na mesma linha.
- Testar todas as **combinações** com uma rainha por linha.
 - Quantas combinações diferentes existem?
 $8^8 = 16.777.216$, equivale a 0,38% de $\binom{64}{8}$

É possível melhorar esse resultado?

BACKTRACKING

- Técnica que examina todas as possibilidades produtivas, abandonando situações com potencial de serem exploradas.

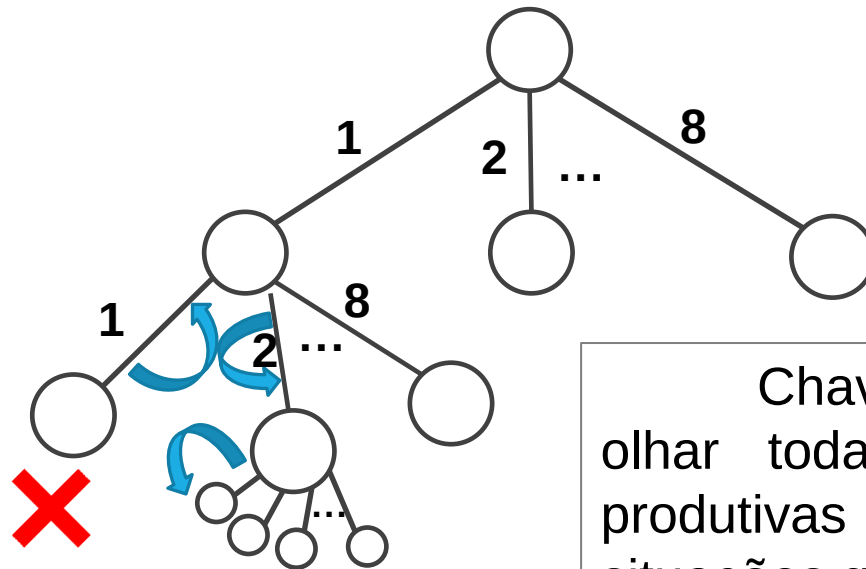
- 8-rainhas



BACKTRACKING

- Técnica que examina todas as possibilidades produtivas, abandonando situações com potencial de serem exploradas.

- 8-rainhas



Chave do backtrack
olhar todas as possibilidades
produtivas e abandonar as
situações que não levam a nada.

BACKTRACKING

- Método batizado de *backtrack* por R. J. Walker por volta de 1950.
- “possibilidades” se referem a diferentes configurações em um espaço de busca.
 - Ex. diferentes formas de colocar as 8 rainhas no tabuleiro.
 - Todas as configurações devem ser geradas de maneira sistemática.

BACKTRACKING

- Na prática, o algoritmo termina na primeira solução encontrada para o problema. Mas aqui, vamos querer como saída do algoritmo todas as soluções.
- Backtracking, basicamente, faz uma exploração parcial ao invés de exaustiva do espaço de busca.
 - As duas propostas (1 e 2) de solução apresentadas percorrem o espaço de busca de forma exaustiva.

BACKTRACKING FRAMEWORK GERAL

- Buscar todas as sequências $x_1, x_2, \dots, x_n$ tal que uma propriedade $P_n(x_1, x_2, \dots, x_n)$ é verdadeira.
 - Onde x_n pertence a um domínio D_k de inteiros.
- Exemplo: problema das 8-rainhas
 - $x_1, x_2, \dots, x_n$: cada x representa uma rainha em uma coluna do tabuleiro.
 - domínio D_k : definido no intervalo $[1, 8]$, ou seja, 1ª coluna, 2ª coluna, e assim por diante.
 - $P_n(x_1, x_2, \dots, x_n)$: as propriedades devem respeitar as restrições das rainhas não se atacarem.

BACKTRACKING FRAMEWORK GERAL

- Exemplo (propriedades)

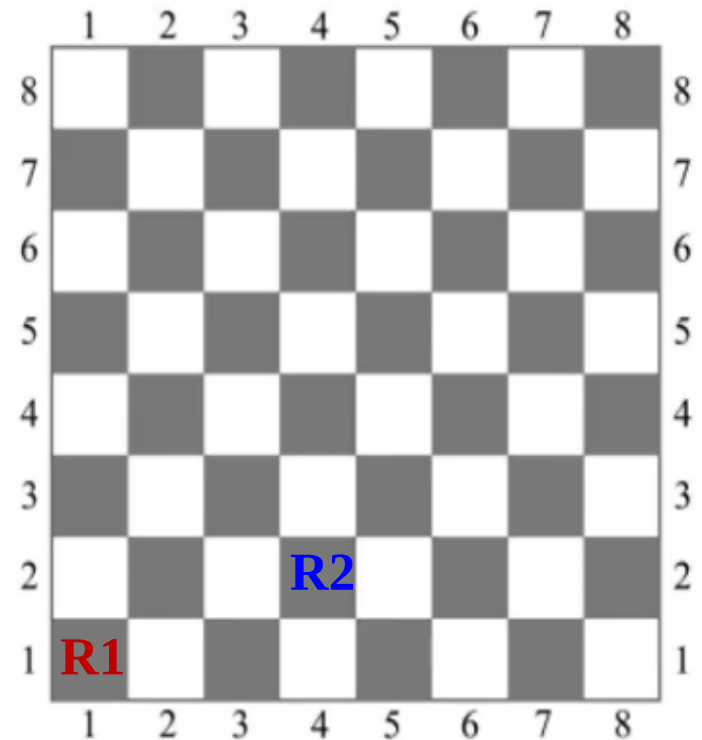
- $P_2(1, 4)$

R1 foi colocada na
linha 1, coluna 1

R2 foi colocada na
linha 2, coluna 4

$$P_2(\textcolor{red}{1}, \textcolor{blue}{4}) = V \text{ ou } F$$

(essa propriedade só será verdadeira
se as rainhas não se atacarem)



BACKTRACKING FRAMEWORK GERAL

- Backtracking, na sua forma básica, envolve o uso da propriedade de ‘corte’ ou ‘poda’ $P_L(x_1, x_2, \dots, x_L)$ para $1 \leq L < n$ tal que:
 - $P_L(x_1, x_2, \dots, x_L)$ é verdadeiro sempre que $P_{L+1}(x_1, x_2, \dots, x_{L+1})$ for verdadeiro.
 - $P_L(x_1, x_2, \dots, x_L)$ é fácil de testar se $P_{L-1}(x_1, x_2, \dots, x_{L-1})$ for verdadeiro.

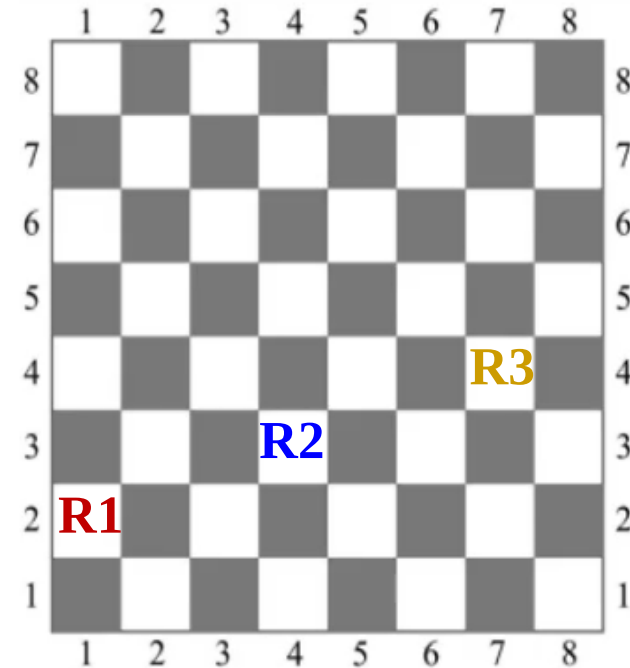
BACKTRACKING – FRAMEWORK GERAL

- $P_L(x_1, x_2, \dots, x_L)$ é verdadeiro sempre que $P_{L+1}(x_1, x_2, \dots, x_{L+1})$ for verdadeiro.

- Exemplo (8-rainhas):

Se $P_3(\textcolor{red}{1}, \textcolor{blue}{4}, \textcolor{brown}{7}) = V$

Então $P_2(\textcolor{red}{1}, \textcolor{blue}{4}) = V$



BACKTRACKING FRAMEWORK GERAL

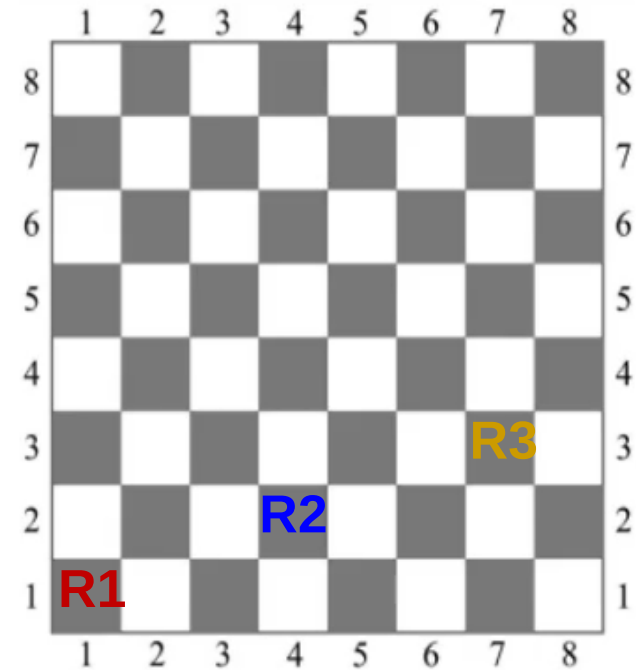
- $P_L(x_1, x_2, \dots, x_L)$ é fácil de testar se $P_{L-1}(x_1, x_2, \dots, x_{L-1})$ for verdadeiro.

- Exemplo (8-rainhas):

$P_3(1, 4, 7)$ é fácil de testar, pois $P_3(1, 4) = V$

Com isso, basta testar se as rainhas:

- $P_3(1, 7)$ se atacam
- $P_3(4, 7)$ se atacam



BACKTRACKING FRAMEWORK GERAL

- Backtracking, na sua forma básica, envolve o uso da propriedade de ‘corte’ ou ‘poda’ $P_L(x_1, x_2, \dots, x_L)$ para $1 \leq L < n$ tal que:
 - $P_L(x_1, x_2, \dots, x_L)$ é verdadeiro sempre que $P_{L+1}(x_1, x_2, \dots, x_{L+1})$ for verdadeiro.
 - $P_L(x_1, x_2, \dots, x_L)$ é fácil de testar se $P_{L-1}(x_1, x_2, \dots, x_{L-1})$ for verdadeiro.

As duas propriedades foram respeitadas no problema das 8-rainhas.

BACKTRACKING

- Função *wrapper*
- Algoritmo recursivo

solucao()

Inicializa P , L e n

<code>busca_solucao</code> (P , L , n)
--

<code>busca_solucao</code> (P , L , n)
--

Se $L > n$

Uma solução P encontrada

Senão

Para todo x_L pertencente ao domínio D_k

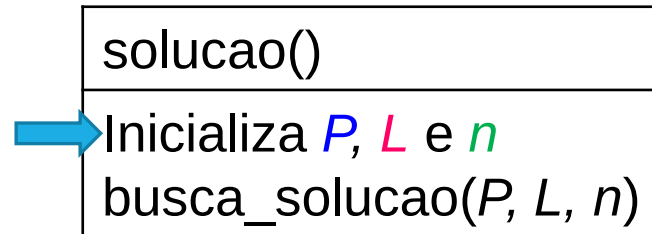
Inserir x_L em P

Se P é verdade

`busca_solucao`(P , $L+1$, n)

Remove x_L de P {Passo de backtrack}

BACKTRACKING



- Problema das 8 rainhas
 - P : vetor para armazenar a posição das 8 rainhas no tabuleiro.
 - L : linhas do tabuleiro [1, 8]
 - n : tamanho do problema, ou seja, $n = 8$ rainhas.

Problema das 4-rainhas

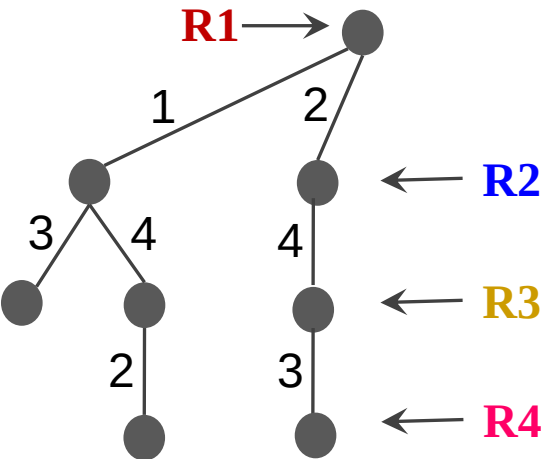
busca_solucão(P, L, n)
Se $L > n$ Uma solução P encontrada Senão Para todo x_L pertencente ao domínio D_k Insere x_L em P Se P é verdade busca_solucão($P, L+1, n$) Remove x_L de P {Passo de backtrack}

solucao()
Inicializa P, L e n busca_solucão(P, L, n)

Problema das 4-rainhas

	1	2	3	4
P	R3	R1	R4	R2

busca_solucacao(P, 4, 4)

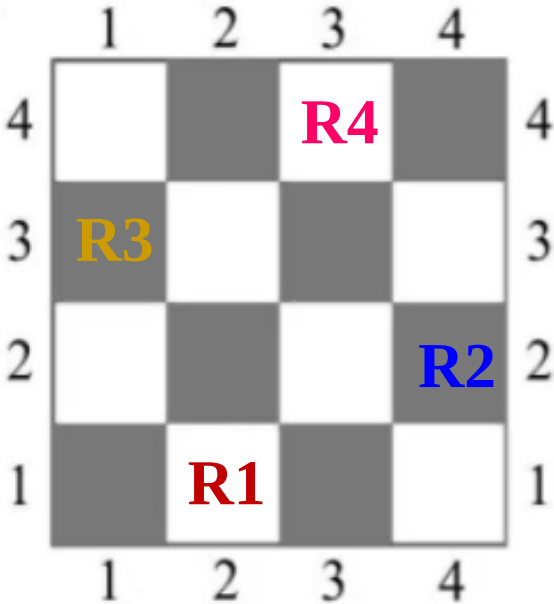


Solução produtiva

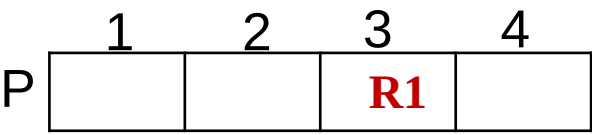
busca_solucacao(P, L, n)

Se $L > n$
Uma solução P encontrada

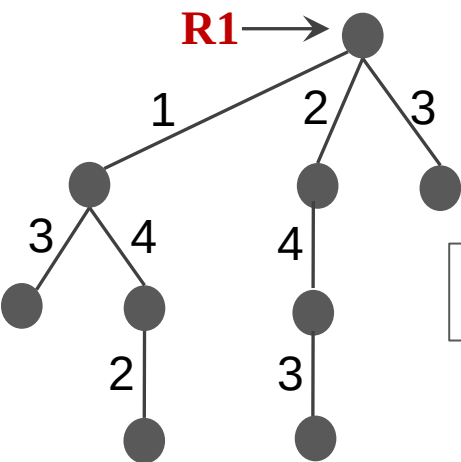
Senão
Para todo x_L pertencente ao domínio D_k
 Insere x_L em P
 Se P é verdade
 busca_solucacao(P, L+1, n)
 Remove x_L de P {Passo de backtrack}



Problema das 4-rainhas



busca_solucacao(P, 1, 4)

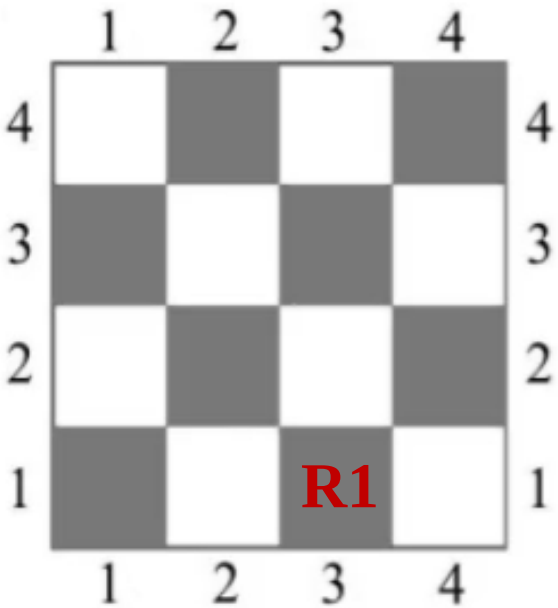


Continuar...

busca_solucacao(P, L, n)

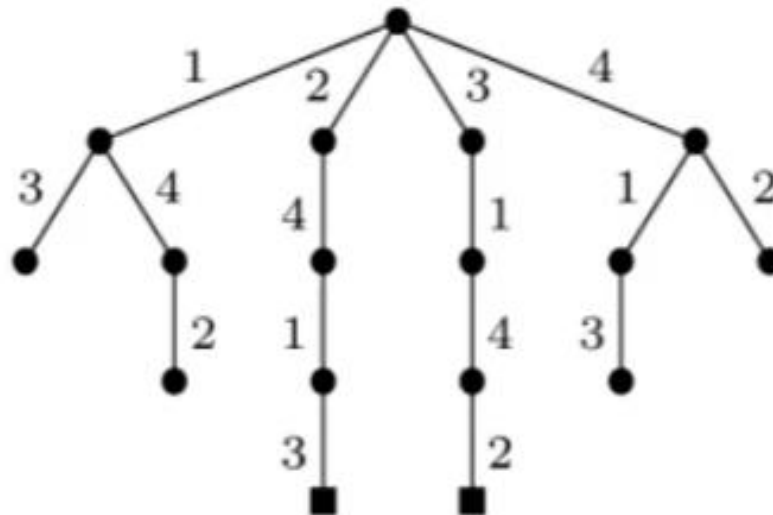
Se $L > n$
Uma solução P encontrada

Senão
Para todo x_L pertencente ao domínio D_k
 Insere x_L em P
 Se P é verdade
 busca_solucacao(P, L+1, n)
 Remove x_L de P {Passo de backtrack}



BACKTRACKING – 4 RAINHAS

- Árvore de busca, ou árvore de backtrack, para 4-rainhas:
 - 17 nós
 - Número de soluções ($P_n = P_4$) é 2.



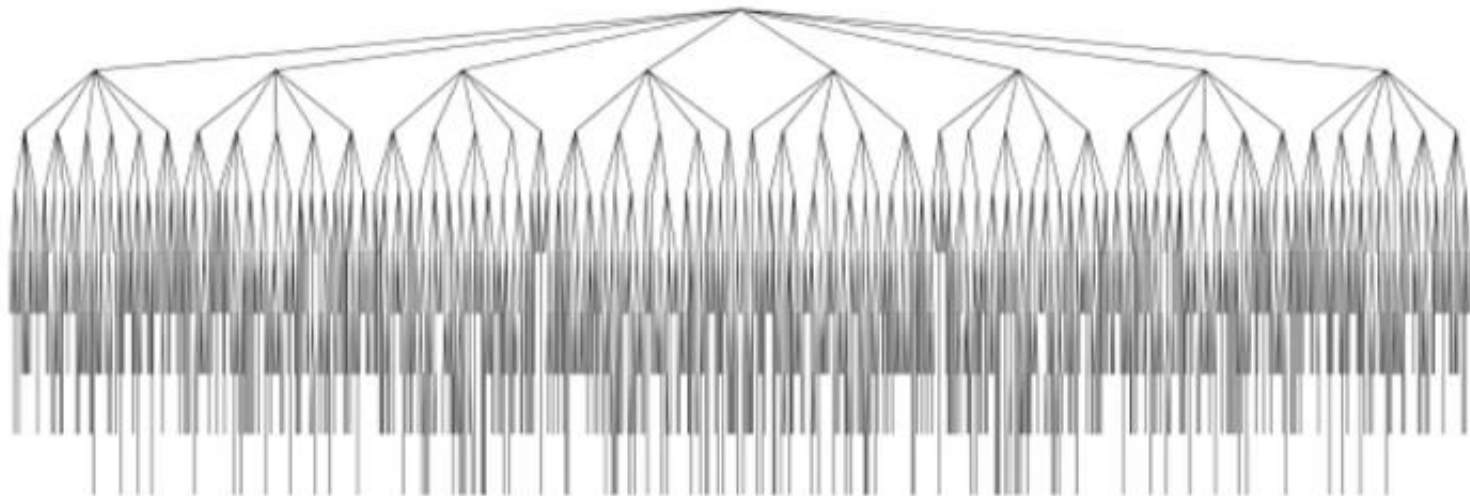
BACKTRACKING – 4 RAINHAS

- Solução Backtrack x Proposta 2

	Solução Backtracki	Solução Proposta 2
Percorreu quantos nós para encontrar as soluções?	17 nós	$4^4 =$ 256 nós

BACKTRACKING – 8 RAINHAS

- Árvore de busca, ou árvore de backtrack, para 8-rainhas:
 - 2057 nós
 - Número de soluções ($P_n = P_4$) é 92.



BACKTRACKING – 8 RAINHAS

- Solução Backtrack x Proposta 2 x Proposta 3

	Solução Backtrack	Solução Proposta 2	Solução Proposta 1
Percorreu quantos nós para encontrar as soluções?	2056 nós	$8^8 =$ 16.777.216	$\binom{64}{8} =$ 4.426.165.368

BACKTRACKING – 27-RAINHAS

- Em 2016, o problema das 27 rainhas foi resolvido (tabuleiro 27 x 27)
 - Um cluster de FPGAs foi usado por 383 dias.
 - 234.907.967.154.122.528 soluções foram encontradas.

BACKTRACKING – N-RAINHAS

- Sendo $Q(n)$ o número de soluções para o problema de n -rainhas temos:

n	0	1	2	3	4	5	6	7	8	9	10
Q(n)	1	1	0	0	2	10	4	40	92	352	724

n	11	12	13	14	15	16
Q(n)	2680	14200	73712	365596	2279184	14772512